
Assignment 1: Ubiquitous Low-power Devices

Deadline: September 30th, 2026; 14:00 CEST

Overview

In the first lecture you saw how low-power devices are a key driver in the world of ubiquitous computing. In this assignment, you will implement software on a low-power microcontroller (MCU) with wireless and sensing capabilities that is powered by a small battery. The constraints on energy and computational power you will encounter will help you experience the challenges in developing such devices. The concrete goals are (1) to understand various low-power strategies when dealing with such MCUs, sensors and wireless communication, and (2) to realize how the energy budget often plays a crucial role in determining the implementation architecture.

The Puck.js Device and Espruino Software

You will use an embedded device called Puck.js, which you can program with the Espruino framework. Espruino lets you program in JavaScript; an interpreter on the device runs your program.

On an embedded device like the Puck.js, energy is consumed by different components – for example, the CPU, the radio electronics, the IO peripherals, sensors, memory chips, etc. You can see a breakdown of the power consumption on the official website ¹. When embedded devices are powered using a limited source of energy, say a battery or a photovoltaic (PV) cell, then your program needs to be constructed keeping the energy constraints in mind. For example, you might have to optimize your code such that its CPU time is minimal, or you need to make judicious use of sensors, LEDs, and network communication. The Puck.js is powered by a CR2032 coin cell (remember from the exercise session how much energy it can deliver?). Such battery-powered devices are often seen in many Wireless Sensor Network (WSN) applications, with both static and mobile deployments. Two well-known examples of WSN applications include animal tracking² and Alpine permafrost monitoring³, among many others.

In the first part of this assignment, you will learn how you can control the Puck.js's power consumption. In the second part, you will compare the energy consumption between two implementation strategies for a problem that involves machine learning.

In the description of each part below, there are several questions posed. Try to answer these without using AI assistance, as it will help you develop the critical thinking required when dealing with challenges of programming low-power devices. You may freely use AI-assisted coding, but please make sure you understand each line of code it produces. Especially because there are several subtle optimizations that coding agents (in their current state at least) seem to miss easily!

Across all tasks in this and the other assignments in this course, you are **required to declare and reflect on** any support that you received from others and, within reasonable bounds,⁴ any support tools that you were using while solving the assignment.

¹<https://www.espruino.com/Puck.js#power-consumption>

²<https://www.princeton.edu/~mrm/sensys04.pdf>

³https://www.parcs.ch/wja/pdf_public/2018/36537_20180605_094754_2007_permasense_talzi.pdf

⁴It is not required that you declare that you were using a text-editing software with orthographic correction; it is however required to declare if you were using any non-standard tools such as generative machine learning models such as GPT

Required Tools

- 1.) A Puck.js which you received from us.
- 2.) Laptop with any standard OS (MacOS, Windows, Linux), **BUT Chrome browser**
- 3.) Install NRF Connect on Laptop. You can find it [here](#).
- 4.) Optional: NRF Connect app on your smartphone. You can use it to receive BLE advertisements and examine the packets.
- 5.) The NRF Power Profiler. See instructions [here](#).

The **Assignment repository** is here in GitHub : [Assignment-1](#). Instructions regarding programming and power measurement are in the repository. The instructions for the tasks and how to hand-in your assignment are detailed in this document.

A Low-power Tilt Sensor for Medical Application

We will incrementally build a small battery-powered sensor (using our Puck.js) that can detect if it has been tilted and, if so, in which direction. The Puck, when moved, should evaluate the direction of tilt and send it out as a payload on BLE advertisement. For practical motivation, let's say this will be a medical diagnostic aid which will determine the quality of limb movements. The goal is to maximize the battery life while satisfying the functional requirement.

Let's divide and conquer. First, you want the device to consume power only when needed. Therefore, in the first part, you will learn how you can keep the sensor in an ultra-low-power mode when idle (i.e., no tilting movement). Then, when the device is moved, we want to use the accelerometer data to evaluate the direction in which the device is tilted. To do this, in the second part you will evaluate two strategies for classifying the tilt – one where a trained Artificial Neural Network (ANN) is deployed on the device, and the other where the ANN is on a host (i.e., your laptop) while the device merely sends data to the host. The question boils down to which is more energy-efficient while being accurate enough.

Since minimizing power consumption is a key focus of this assignment, you have to measure and, more importantly, **reason about the observations**. For each of the tasks defined in the following sections, you will use the NRF Power Profiler to measure the power consumption. In many places, you will be asked to first estimate the energy consumption before measuring. This requires that you analyze the manufacturer's datasheet of the MCU and sensors to know its power consumption characteristics. You might find the description [here](#) and [here](#) useful.

You need to deliver (a) working code for Puck.js and in Part 2, also the code that trains the neural network on your PC, (b) Power Profile export data where you are required to measure the power consumption, (c) estimations and interpretation of power consumption and answers to the questions posed. The assignment repository contains template folders for files and a template format for written responses – please follow these strictly.

Part 1: The right way to sleep

Let's begin with a rather simple implementation. You need to continuously observe the accelerometer and send the data as a BLE advertisement payload. The accelerometer values are in mG units. To avoid handling floats in the BLE payload, you can use the following conversion to get a nicely scaled integer:

```
var a_x = ( accel.acc.x / 82) ;  
var a_y = ( accel.acc.y / 82);  
var y_z = ( accel.acc.z / 82);
```

Task 1 (P1.T1): The battery killer Write a program that measures acceleration along the X, Y, Z axes (a_x, a_y, a_z) every 100ms and advertises the value (i.e., BLE Advertisement consisting of 12 bytes (3 x 4-byte floats)). After you download the program, you can observe the advertisements on NRF Connect or the test program provided in the assignment repository. First, estimate, and then measure the power consumption of your implementation and report your observations.

P1.Q1: How long would the CR2032 last? Explain your calculation.

Can we do better? We expect the Puck to be moved only once in a while. This means we don't need to continuously measure the acceleration. Perhaps we can have the Puck be cyclically "active" for a while and then "sleep" for a while. Fortunately, the NRF52 MCU can enter different sleep modes. The *deepest* is through the function call `NRF.sleep()`, which suspends the MCU but keeps the BLE radio alive. Then, in addition to sleep, you call `NRF.setAdvertising()`, which will also stop the BLE radio. This effectively shuts down the MCU. Therefore, when you put an MCU to sleep, you better have a plan to wake it up. Otherwise you have no access to it to download a new program. The only way to wake up a sleeping MCU is to use an *interrupt*. For example *timer* or a signal from a sensor. Let's use a timer periodically wake up our Puck.

Task 2 (P1.T2): Periodic wake-up Write a program that will setup a timer to "wake up" the Puck.js every 5s, then read the accelerometer and advertise the 12 bytes on BLE every 100ms for 5 seconds before going back to sleep⁵. Once again, first estimate, and then measure the power consumption.

P1.Q2: We are now effectively active only half the time. But why is the energy consumption for a 10s cycle not half of what you saw in P1.T1?

Not surprisingly, the power consumption has reduced compared to P1.T1. But then we might lose movements that can happen during the 5s sleep period. Right? It would be nice to wake up when required - i.e, wake up movement is sensed.

A life-saving feature of MCUs is the possibility of *external interrupts*. You can use such an interrupt to wake up a sleeping MCU. Modern sensing chips, like the accelerometer (STMicroelectronics LSM6DS3TR-C) on the Puck.js are in fact tiny MCUs in themselves. The accelerometer can be told to "watch out" for motion and only then "poke" or *interrupt* the main (NRF52) MCU. In this way, we won't lose movements that happen during sleep.

Task P1.T3: Wake on shake Implement code that will wake up the sleeping Puck when it is moved. It should then obtain the accelerometer values every 100ms and send it out as BLE advertisements (like in P1.T2). After 5 seconds of doing this, it should go back to sleep (only to be woken if it is shaken). There are several ways to set up the accelerometer to wake up the MCU when it detects movement. A straightforward way is to use the convenience library described [here](#).

As usual, first try to estimate and then measure the power consumption. P1.Q2: (a) Is there a difference between sleep-phase current and the current profile? If so, why? (b) How long (days/months/years) will the CR2032 battery last if the puck is moved 10 times a day?

Ok, now that you have a way to energy-efficiently get the accelerometer values when needed, let's find a way to classify the *pose* of the device that can be derived from the accelerometer data.

⁵The practical reason for this 5s wait-before-sleep is that we want the Puck.js to be listening for a new program upload request from the Web IDE during this. Otherwise we get "locked out", or what is commonly known as "bricked"

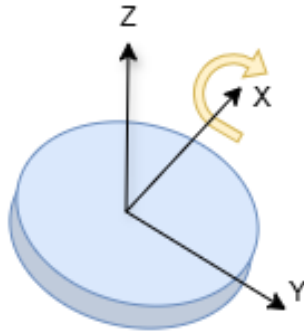


Figure 1: You will train a neural network to recognize the direction in which the puck has been tilted about an axis

Part 2: The question of locality

Let us now try to classify some simple orientations of the Puck. To keep the complexity low, we would like to only detect if the Puck has been tilted (a) left, (b) right, or (c) kept straight.

But to keep things a bit energy-challenging, let us use an artificial neural network (ANN) to do this classification. Well, from your knowledge of machine learning, you might critically think if this use-case *really* needs a neural network. Still, just for the sake of having a computation-intensive model, let us go for it. Now, it is quite usual that the network would be trained on a relatively capable computing device (like your laptop). But then *where* should the trained model run? Let us consider two variants:

- *On-board* deployment on the embedded device, such that the accelerometer data is fed directly to the ANN that is running on the Puck.js, and only the *results* need to be transmitted on BLE.
- *Off-board* deployment, such that the ANN is hosted on an external computation host (e.g., your laptop), and the Puck uses its communication interface (BLE) to transmit the *raw data* to the off-board host device (where the classification is done).

We are concerned with how much energy the embedded device requires in each variant, and how the energy profiles look like. This is an important question when maximizing the lifetime of battery-powered devices since the energy budget of such devices is limited. In the first variant, the embedded device has to locally carry out the computation task of the ANN model, whereas in the second variant it has to send raw data over a wireless link (BLE) – depending on the device and application, either of these might be more energy-hungry. You will implement both versions on the Puck.js and contrast the two approaches.

Task: Prepare training/test data Use the code provided in the assignment repository to receive the accelerometer data. You need to flash the Puck.js with the `data-sender.js` code and run the Python code `data-reciever.py` on your laptop. In the Python code you have to first adjust the name of the Puck and the filename (both marked with comments). Then start the Python program and wait for it to connect to the Puck. Twist the Puck right or left and press the button; hold for about 10s while varying the tilt angle, and then release the button. Press Ctrl-C to stop the Python program. You will see `right.csv` / `left.csv` files in the same directory.

Task P2.T1: Train network Now the data obtained above can be used to train and test your neural network. You are free to use any framework. We recommend SciKit because of its simplicity. The ANN trainer should finally output weights and biases of the network which you can use on the Puck.js. Report the accuracy of prediction achieved by your network.

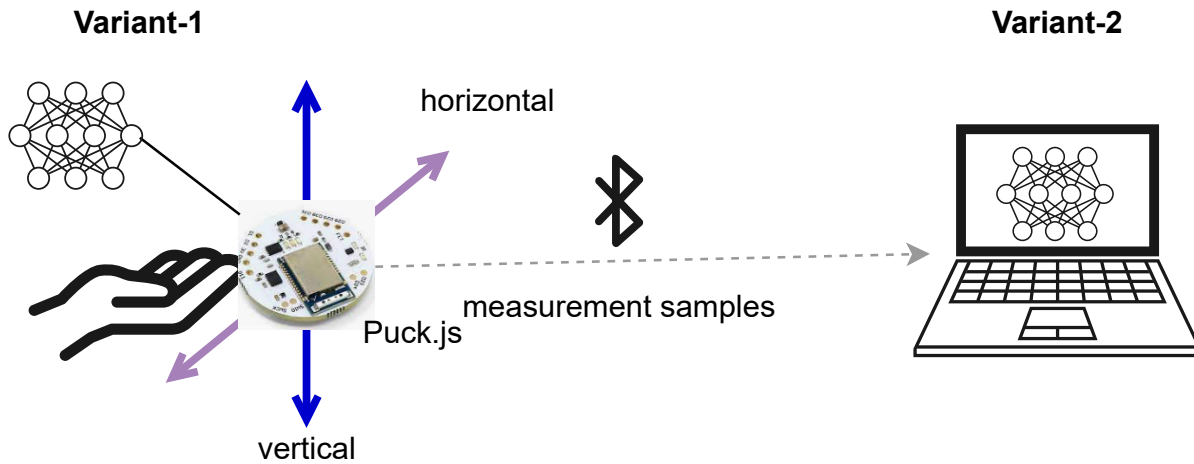


Figure 2: Which one would be more energy efficient – running the ANN on-board (Variant 1) or off-board (Variant 2)?

Task P2.T2: Deploy on Puck Create a program for the Puck that will use the trained network (in form of weights and biases) to classify the orientation. The program should get the instantaneous accelerometer values and feed-forward it through the ANN and output the prediction as 0=left, 1=right, and 2=straight. The prediction (the single byte) should be sent as BLE advertisement payload for 5 seconds before the Puck goes back to sleep. Measure power consumption and report your observations.

Task P2.T3: Offboard the model You will now send the accelerometer data from the Puck to a receiver program on your PC. The Puck's program will now be what you created in P1.T3 (i.e., a program that wakes up on movement and sends out the 12-byte accelerometer data). The receiver program needs to have two parts: one which receives the BLE advertisements and the next which takes in the accelerometer readings from the payload and feeds it to the trained network. You can reuse the `utilities/host-ble-adv-receiver.py` code provided in the assignment repository to build the first part. The second part should feed the received data through the network you trained in the previous step and output the classification on the console. Once again, you are free to choose the framework for your host-based ANN program. Measure the power consumption on the puck.

P2.Q1: Explain the difference or lack of it in power consumption between P2.T2 and P2.T3.

Optional P2.Q2: At what point would the situation with energy consumption change? i.e., the lower power-consuming variant becomes the larger one and vice-versa. In answering this question, justify by explaining how the energy cost would change when the complexity of the ANN increases or quantity/quality of required input data changes. If the situation is unlikely to change, explain why. You can also examine the energy consumption for the part of the code that executes the ANN model in the on-board variant, and the energy required to send the data over BLE (to the host) — the assignment repository contains description of how to do this. Estimate and/or experiment to see how this will change when the network is larger or more than one sample of accelerometer data is required to be sent.

Hand-in and Grading

The evaluation of this assignment is **binary**. The requirements for passing the assignment are:

- 1.) Submission of working code for **each** of the tasks. It should be possible to deploy your code and verify its functioning.
- 2.) Submission of Power Profiler files that correspond to your implementation. Where you are asked to

measure the power consumption, your exported PPK2 file should contain the data for at least one entire cycle of operation. The parts that are visible on the profile should be reference to in your responses. You may use screenshots from the Power Profiler here to make it clear.

- 3.) You should provide a reasonable interpretation of the power consumption results. Your description should be concise by highlighting where and what happened in a cycle. Example: "Immediately on wake-up, the Puck.js consumed 5mA steadily for 150ms. The following screenshot shows this. Our interpretation is that this is a bootstrapping overhead"
- 4.) The answers to the posed questions should be presented objectively (the questions are very objective too). For example: "Despite sleeping for half the time, the average power consumption is not halved because the MCU has to do ... which takes about ...mA ".

By the deadline, please upload your content as ZIP-file to Canvas, or put a link to your GitHub repository in Canvas and make sure it is accessible to Ganesh (<https://github.com/codepasta>) and Jannis (<https://github.com/jrstrecker>). If you upload a ZIP file, name it 'UbiComp-Assignment01-<your last name>-Group<Your group number>.zip'. If you prefer to host your code in a GitHub repository, name the repo 'UbiComp-Assignment01-<your last name>-Group<Your group number>'. Example: UbiComp-Assignment01-Muster-Group4